Logan Barnett's Résumé

Contact

Email logustus@gmail.com

Mobile 602.264.3584

Blog `https://blog.logustus.com`

Github `https://github.com/loganbarnett`

LinkedIn `https://www.linkedin.com/in/logan-barnett-2556114/`

Location Beaverton, Oregon, USA (remote)

Summary

22 years as a professional software engineer, spanning video games, embedded systems, web applications, and the tooling that engineering and operations teams depend on. My current focus is systems programming in Rust, building reproducible tooling with NixOS and nixpkgs, and applying LLMs as deliberate engineering tools — including a practical understanding of where they fail and how to design workflows that account for that. I care about sustainable software: strong type systems, thorough testing, and documentation that makes code legible to the next engineer. I enjoy mentoring and gravitate toward the hard problems.

Job Experience

Houghton Mifflin Harcourt (HMH, formerly NWEA) (2023–current)

Embedded software engineer on the DevOps team — the person who builds the tools the team runs on, rather than running them. The team manages over 2000 VMs via Puppet and r10k; my role is the software layer above that: deployment orchestration, environment management tooling, and the internal systems that let the team operate at scale without coordinating by hand.

1. Took ownership of Environment Down Days (EDD) — a recurring operational process that had grown into organized chaos: 10 engineers working in parallel for 3 days, each with their own bespoke scripts. Recognizing that EDD and the existing deployment tooling were solving the same problem twice, I unified them, paid down significant tech debt, and introduced parallelism where it was safe. Result: 10 engineers × 3 days collapsed to 1 engineer × 1–1.5 days, with a system that is correct and maintainable rather than just fast.
2. Mentored 4–6 engineers at a time on the Puppet ecosystem, deployment workflows, and automation practices.

3. Migrated the primary internal tooling suite (nwea-gem) and extracted libraries from it to CICD, and converted approximately a dozen Jenkins freestyle jobs to Groovy-based Jenkinsfiles.

4. Used LLMs extensively for troubleshooting, code edits, and automating repetitive engineering tasks via CLI.

Principal Instructor at Alchemy Code Lab (2022–2023)

Taught 3–4 cohorts of roughly 20 students each across a year at a code school that had transitioned permanently to remote delivery following Covid.

1. Taught advanced JavaScript topics: Promises and async patterns, type safety, reducers, and Data Structures and Algorithms.

2. Stressed simplicity over ease — a meaningful distinction when teaching React. A recurring focus was guiding students to recognize when to step back, refactor, and reach for reducers or functional patterns instead of defaulting to `useState`.

3. Mentored students individually on software engineering practices beyond the curriculum, including how to think about maintainability and code that the next person can understand.

NWEA (2016–2022)

Hired for JavaScript expertise; spent the first several years on frontend and application engineering before transitioning to DevOps at the onset of the COVID lockdown in 2020.

DevOps (~2020–2022)

Joined entirely remote — interviewed on-site but never worked there in person. Brought a software engineer's perspective on what it actually feels like to consume DevOps tooling, which shaped a lot of the improvements made.

1. Assumed stewardship of the primary internal Ruby tooling suite (nwea-gem), which controlled colo management, deployments, release planning, and credential synchronization. Improved reproducibility, test coverage, memoized API calls, and automated deployments to keep the suite functional in a rapidly changing environment.

2. Directed the Build Engineering sub-team's effort to consolidate the build infrastructure: from ~8 largely unmanaged VMs (engineers installing tools manually as needed) to the Cloud Swarm — a flexible pool of Dockerized Jenkins agents under proper configuration management.

3. Maintained and extended the Nomad scheduling infrastructure that ran in-house deployment and management tools (Kraken, Compass) alongside approximately a dozen engineer-built applications.

Frontend and Application Engineering (~2016–2020)

Senior engineer leading the build of a new React-Redux proctoring interface for running student tests at scale — approximately 100,000–200,000 concurrent users at peak, where AWS API limits were a genuine constraint.

1. Achieved 100% Flow type coverage across the UI, treating the type system as a first-class testing tool rather than an afterthought.

2. Built a suite of 350+ Cucumber + Nightwatch.js acceptance tests that QA members could author and maintain without deep engineering knowledge. The suite ran in ~50 minutes on feature branches; a `--ff-only` merge policy meant master was bit-for-bit identical to the branch and did not need to be retested.

3. Built a feature flag system from scratch — deliberately simple, local to the UI, and fully integrated with the test suite. Tests declared which flags they required via Cucumber tags, so every feature launched with "flag off" tests already in place before "flag on" tests were added.

4. Kept API mocks automatically synchronized with Swagger definitions, making the UI a reliable enforcer of API contracts and catching backend drift before it reached production.

UTi Worldwide / DSV (via IT-Motives) (2014–2016)

Hired specifically as AngularJS expertise to lead the UI of a customer-facing shipment tracking portal at a global supply chain coordinator. Tenure ended when DSV acquired UTi and shut down the Portland office.

1. Restructured the AngularJS application architecture to support a growing UI surface area; established TDD on the team and brought the UI to 100% test coverage.

2. Set up Jenkins for automated CI with test coverage reporting; introduced Cucumber for executable business requirements.

3. Led a React + Redux pilot project (self-service identity management against Oracle OIM/OID) intended to demonstrate a path for modernizing legacy applications incrementally.

4. Served as informal AngularJS consultant to other UTi teams evaluating the framework.

Early Career (2004–2014)

Software engineering roles spanning educational games, hosted services, Rails consulting, embedded device tooling, and enterprise .NET:

- *Arizona State University → E-Line Media (2011–2014)* — Co-lead and senior developer on Atlantis-Remixed, an educational game built for academic research on the efficacy of video games as a teaching medium. Led a team of 4 engineers; built a 2D visual scripting tool for designers; designed the build and patching systems in Unity / .NET / Mono; integrated with a Ruby on Rails backend.

- *GoDaddy (2010–2011)* — Hosted Exchange: migrated management from the Outlook API to PowerShell; built internal support tooling in MVC3; extensive PowerShell work against Exchange 2010.

- *Integrum Technologies (2009–2010)* — Rails consulting shop. Pair-programmed across client projects including fantasy sports backends, a legacy Rails app with thousands of users, and a public transit schedule system.

- *Happy Camper Studios (2007–2009)* — Built configuration and diagnostics desktop applications for satellite modems (Radyne / Comtech). The scale of the UI directly drove the creation of Monkeybars, a JRuby-based MVC framework for Swing applications, which was later generalized and open-sourced.

- *UHaul International (2004–2007)* — First engineering position out of DeVry. .NET desktop applications, SOAP web services, .NET remoting, and a Java/.NET integration layer for a self-insurance claims system.

Full details available in the long-form CV.

Proficiencies

Javascript React, Redux, Three, Angular, Node, ES6, Webpack, Flow, npm, yarn

Rust Actix, futures, abstract data types, monad chains, and avoiding `unwrap` ;)

Nix NixOS, nixpkgs, home-manager, nix-darwin, flakes, declarative system configuration, upstream nixpkgs contributions

Ruby Ruby on Rails, Sinatra, JRuby, Cucumber, RSpec

Java Swing, JAX-RS, JRuby, JUnit, SNMP4J

.net C#, Boo, Unity/Mono, MVC, WCF, NUnit, OData, Powershell

Infrastructure and Ops Puppet (Hiera, r10k), Docker, Nomad, NATS, Jenkins

Databases PostgreSQL, MongoDB, MySQL, SQL Server, Oracle + PL/SQL

LLMs Agentic workflows, MCP (Model Context Protocol), prompt engineering, tool use, Claude

Misc git, Perforce, svn, hg, Plastic SCM, vim, emacs, literate programming, functional programming, JIRA, Pivotal Tracker, Trello

Selected Side Projects

1. dns-smart-block — Network-level DNS content blocking informed by LLM classification. Rust workspace deployed via NixOS flake.
2. garage-queue — Capability-aware work queue in Rust. Workers advertise capabilities; producers submit payloads; routing logic lives entirely in configuration — the queue itself stays workload-agnostic.
3. sonify-health — Turns infrastructure health into ambient sound, so operators can stay aware without watching a dashboard. Written in Rust.
4. nix-config — Declarative Nix configuration for an entire self-managed home network; every host defined as code, deployed with a custom copy-closure tool.
5. sytter — Event-driven host automation in Rust. TOML-defined triggers, conditions, and actions; cross-platform.
6. Lambda Cast — Co-hosted ~20 episodes on functional programming concepts (2016–2019).

Education

DeVry University — BS in Computer Engineering Technology, 2004.